Convex Hull Trick

Tối ưu hoá DP bằng đường bao lồi

Trường Phổ thông Năng khiếu – ĐHQG TP.HCM

14/04/2026

Mục lục

1. Đặt vấn đề
2. Bài 1: Mở quà – CHT cơ bản (min DP)
3. Bài 2: Mỏ quặng – CHT max DP
4. Bài 3: Ván gỗ – Tiền xử lý và CHT
5. Bài 4: Tia và đường thẳng – Bao lồi offline
6. Tổng kết

DP chuyển trạng thái $\mathcal{O}(N^2)$

Dạng bài quen thuộc

$$\mathrm{dp}[i] = \min_{j<i}\{\, \mathrm{dp}[j] + \mathrm{cost}(j,i) \,\}$$

- N trạng thái, mỗi trạng thái duyệt $\mathcal{O}(N)$ tiền trạng thái.
- Tổng: $\mathcal{O}(N^2)$ phép tính.

Vấn đề khi N lớn

$N = 2 \times 10^5 \;\Rightarrow\; N^2 = 4 \times 10^{10}$ phép tính $\Rightarrow$ **TLE**

Khi nào tối ưu được?

Khi $\mathrm{cost}(j,i)$ có **cấu trúc tuyến tính** theo j và i, ta có thể coi mỗi j là một **đường thẳng** và mỗi i là một **truy vấn**. Đây chính là ý tưởng của **Convex Hull Trick**.

Kiến thức nền: Đường thẳng $y = mx + b$

Các khái niệm cơ bản

- **Slope (hệ số góc)** m: độ dốc, $m > 0$ tăng, $m < 0$ giảm.
- **Intercept** b: giá trị tại $x = 0$.
- **Giao điểm** hai đường $\ell_1 : y = m_1 x + b_1$ và $\ell_2 : y = m_2 x + b_2$:

$$x^* = \frac{b_1 - b_2}{m_2 - m_1} \quad (m_1 \neq m_2)$$

Khi nào ℓ_1 tốt hơn ℓ_2?

- Với $x < x^*$: đường nào có slope *lớn hơn* cho y nhỏ hơn (nếu $m_1 < m_2$, thì ℓ_1 thắng bên trái).
- Với $x > x^*$: đường còn lại thắng.
- **Ý tưởng CHT**: mỗi đường chỉ "thắng" trên một *đoạn* của trục x.

Ví dụ số

$\ell_1 : y = -2x + 8$, $\ell_2 : y = x + 2$

$$x^* = \frac{8 - 2}{1 - (-2)} = 2$$

$x < 2$: ℓ_1 nhỏ hơn $\quad$ $x > 2$: ℓ_2 nhỏ hơn

Convex Hull Trick – Ý tưởng cốt lõi

Bài toán tổng quát

Cho N đường thẳng $\ell_j : y = m_j x + b_j$. Với mỗi truy vấn x_i, tìm:

$$\min_j \{m_j \cdot x_i + b_j\}$$

Liên hệ với DP

Nếu $\mathrm{dp}[i] = \min_{j<i}\{\mathrm{dp}[j] + A(j) \cdot B(i) + C(j)\}$, hãy đặt:

$$m_j = A(j), \quad b_j = C(j) + \mathrm{dp}[j], \quad x_i = B(i)$$

Khi đó: $\mathrm{dp}[i] = \min_j \{m_j \cdot x_i + b_j\}$

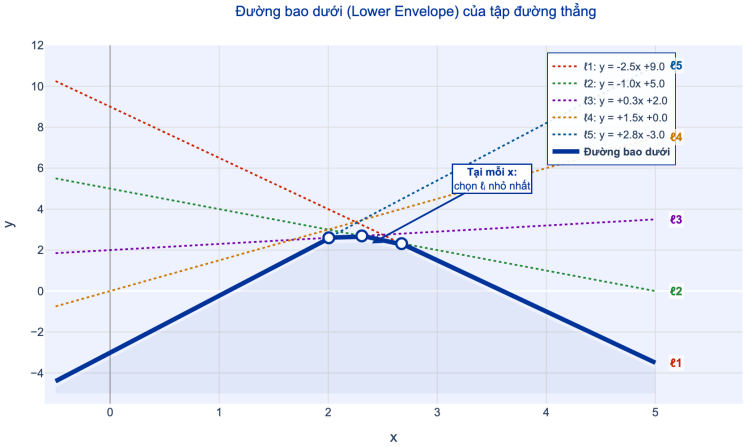
Đây chính xác là bài toán đường thẳng!

Câu hỏi then chốt

Trong N đường thẳng, liệu có đường nào *không bao giờ* là tối ưu không?

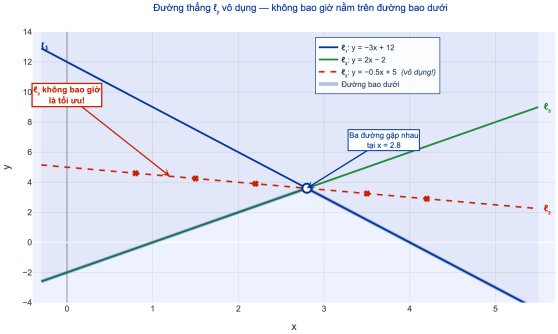
$\Rightarrow$ Xem slide tiếp theo!

Đường bao dưới – Lower Envelope



Các đường không nằm trên đường bao dưới $\Rightarrow$ **không bao giờ tối ưu** $\Rightarrow$ có thể **loại bỏ hoàn toàn**!

Đường thẳng "vô dụng" – không bao giờ là tối ưu



Tiêu chí loại bỏ

Ba đường ℓ_1, ℓ_2, ℓ_3 (slope tăng dần). ℓ_2 bị loại nếu:

$$x^*(\ell_1, \ell_3) \;\leq\; x^*(\ell_1, \ell_2)$$

tức là ℓ_3 "vượt qua" ℓ_1 trước khi ℓ_2 kịp vượt.

Hệ quả

Chỉ cần giữ những đường tạo thành **bao lồi** – mỗi đường thẳng trên đúng một đoạn liên tiếp của trục x.

Bài 1: Mở quà – Đề bài

Bài toán

Có N món quà xếp thành dãy, giá trị của nhóm $[j+1 \dots i]$ là $(S_i - S_j)^2$ (trong đó $S_k = a_1 + \cdots + a_k$). Chia dãy thành các nhóm liên tiếp để **tổng chi phí nhỏ nhất**.

Công thức truy hồi

$$\mathrm{dp}[i] = \min_{0 \le j < i} \Big\{ \mathrm{dp}[j] + (S_i - S_j)^2 \Big\}, \quad \mathrm{dp}[0] = 0$$

Ràng buộc

$N \le 2 \times 10^5 \quad \Rightarrow \quad$ Cần $\mathcal{O}(N)$ hoặc $\mathcal{O}(N \log N)$, không được $\mathcal{O}\big(N^2\big)$.

Giải $\mathcal{O}(N^2)$ – Quá chậm!

```
for (int i = 1; i <= n; i++) {
    dp[i] = INF;
    for (int j = 0; j < i; j++)
        dp[i] = min(dp[i], dp[j] + (s[i]-s[j])*(s[i]-s[j]));
}
```

Kết quả:

$N = 2 \times 10^5 \Rightarrow 4 \times 10^{10}$ phép tính. Máy tính làm khoảng 10^8–10^9 phép/giây $\Rightarrow$ **chắc chắn TLE**.

Câu hỏi

Công thức $(S_i - S_j)^2$ có cấu trúc gì đặc biệt?
Hãy **khai triển** và quan sát!

Bước 1 – Khai triển đại số

Khai triển $(S_i - S_j)^2$

$$\begin{aligned}\mathrm{dp}[i] &= \min_{j<i}\{\mathrm{dp}[j] + S_i^2 - 2S_iS_j + S_j^2\} \\ &= S_i^2 + \min_{j<i}\{\underbrace{\mathrm{dp}[j] + S_j^2}_{b_j} + \underbrace{(-2S_j)}_{m_j} \cdot \underbrace{S_i}_{x}\}\end{aligned}$$

Nhận ra dạng đường thẳng

- $m_j = -2S_j$ (slope, chỉ phụ thuộc j)
- $b_j = \mathrm{dp}[j] + S_j^2$ (intercept, chỉ phụ thuộc j)
- $x_i = S_i$ (query, chỉ phụ thuộc i)

$$\boxed{\mathrm{dp}[i] = S_i^2 + \min_{j<i}\{\underbrace{m_j \cdot x_i + b_j}_{\ell_j(x_i)}\}}$$

Mỗi j cho một **đường thẳng** ℓ_j. Bài toán quy về: với mỗi x_i, tìm ℓ_j nhỏ nhất.

Bước 2 – Tính phương trình đường thẳng (Ví dụ $a = [1, 2, 3, 4]$)

$S = [0, 1, 3, 6, 10], \quad \mathrm{dp}[0] = 0$

j	S_j	$\mathrm{dp}[j]$	$m_j = -2S_j$	$b_j = \mathrm{dp}[j] + S_j^2$
0	0	0	0	0
1	1	1	-2	2
2	3	5	-6	14
3	6	14	-12	50

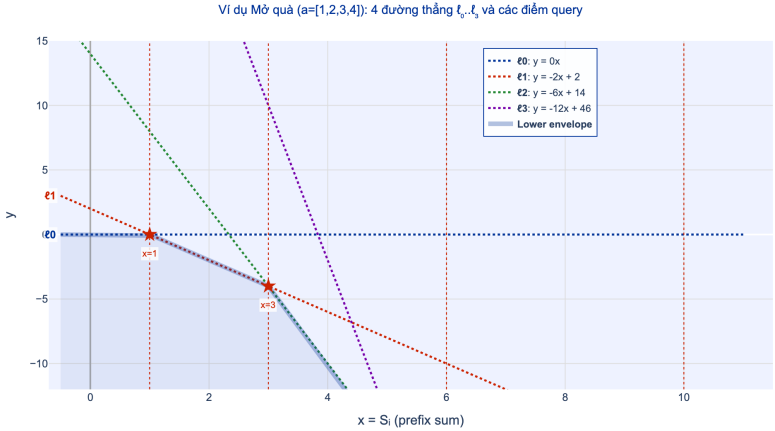
Bốn đường thẳng

- $\ell_0 : y = 0$
- $\ell_1 : y = -2x + 2$
- $\ell_2 : y = -6x + 14$
- $\ell_3 : y = -12x + 50$

Các điểm query

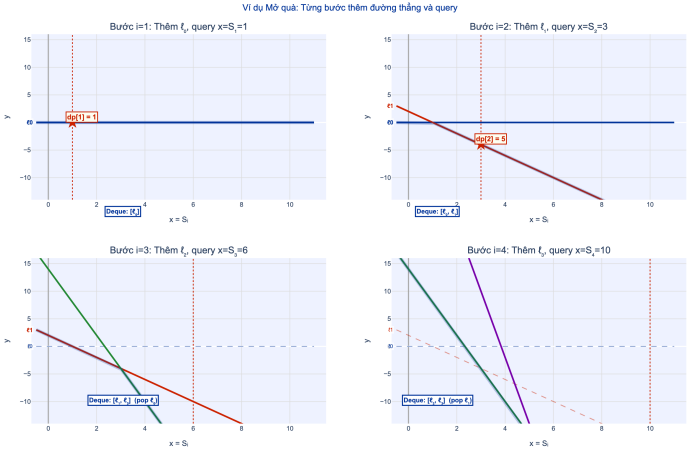
- $i = 1$: query $x = S_1 = 1$
- $i = 2$: query $x = S_2 = 3$
- $i = 3$: query $x = S_3 = 6$
- $i = 4$: query $x = S_4 = 10$

Bước 3 – Minh hoạ 4 đường thẳng và điểm query



Tại mỗi query $x = S_i$: $\mathrm{dp}[i] = S_i^2 + \min_j\{\ell_j(S_i)\}$ — chọn đường thấp nhất.

Bước 4 – Từng bước thêm đường và tính DP



Mỗi bước i: thêm ℓ_{i-1}, query tại $x = S_i$, lấy min $\Rightarrow \mathrm{dp}[i]$. Đường tối ưu **dịch phải** khi x tăng.

Bước 5 – Tại sao query dịch phải? Dùng Deque!

Nhận xét 1: Slope đơn điệu

$m_j = -2S_j$ giảm dần (vì S_j tăng):

$$m_0 \geq m_1 \geq m_2 \geq \cdots$$

Đường thêm vào theo **slope giảm dần**.

Nhận xét 2: Query đơn điệu

$x_i = S_i$ tăng dần:

$$x_1 \leq x_2 \leq x_3 \leq \cdots$$

Query đến **từ trái sang phải**.

Hệ quả: Deque $\mathcal{O}(N)$

Khi query tăng, đường tối ưu chỉ dịch phải.

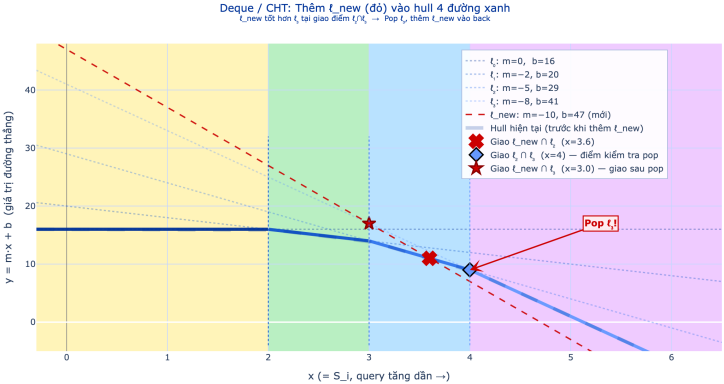
- Không quay lại đường đã bỏ.
- Pop front khi đường thứ 2 tốt hơn.
- Mỗi đường push/pop **1 lần**.

$\Rightarrow$ **Tổng O(N)**.

Từ ví dụ $a = [1, 2, 3, 4]$:

$i = 2$: ℓ_0 bị pop ($x = 3$: ℓ_1 tốt hơn)
$i = 3$: ℓ_1 bị pop ($x = 6$: ℓ_2 tốt hơn)

Deque trên mặt phẳng – Thêm đường mới, Pop back



4 vùng màu = 4 đoạn x mỗi đoạn có một đường xanh thẳng (bao lồi dưới hiện tại).
Khi thêm ℓ_{new} (đỏ): kiểm tra tại giao điểm $\ell_2 \cap \ell_3$ – nếu ℓ_{new} tốt hơn ℓ_3 tại đó $\Rightarrow$ **pop** ℓ_3 **ra khỏi back** trước khi push.

Deque – Duy trì đường bao dưới động

Ví dụ minh hoạ

$n = 5,\ w = [2, 1, 3, 2, 2],\ C = 10.$ Prefix sum: $S = [0, 2, 3, 6, 8, 10]$.

Đường ℓ_j: $m_j = -2S_j,\ b_j = \mathrm{dp}[j] + S_j^2$. Query tại $x_i = S_i$ (tăng dần).

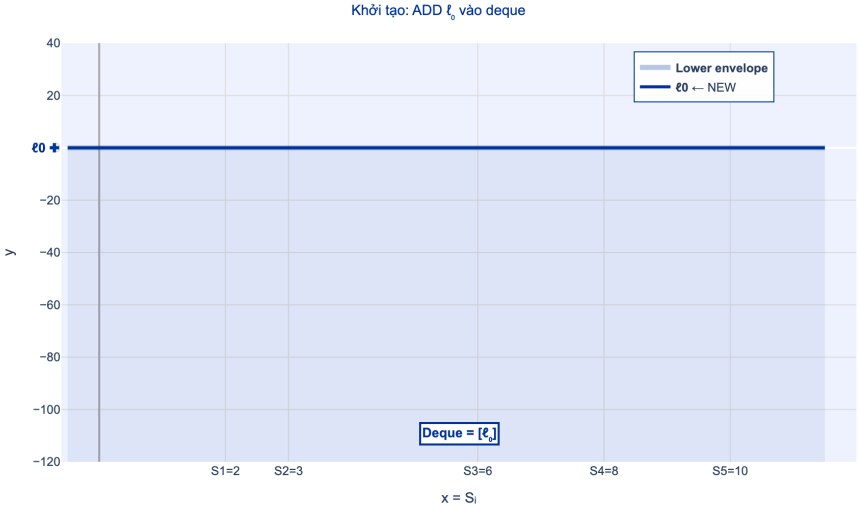
Hai thao tác deque

- **Push back**: thêm đường mới vào cuối. Loại đường cuối trước nếu nó bị che.
- **Pop front**: khi query x tăng, đường đầu không còn tốt nhất → bỏ đi.

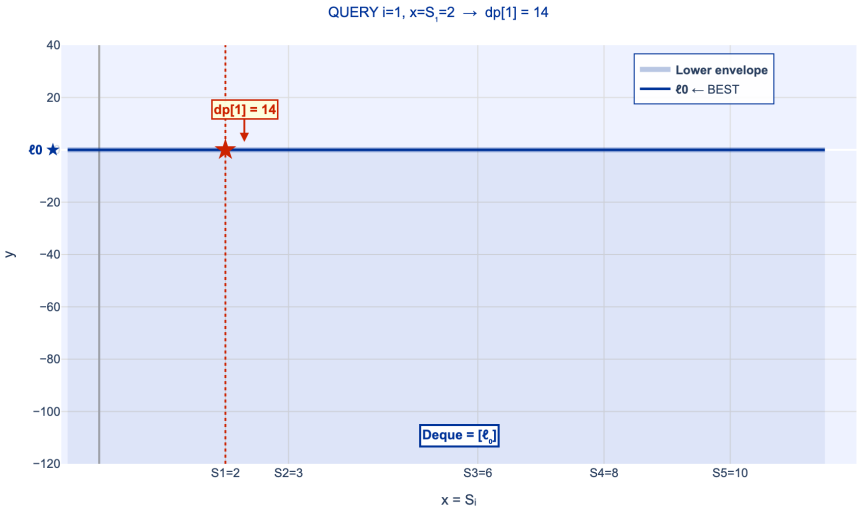
Tóm tắt 10 bước

Add ℓ_0	Deque $= [\ell_0]$
Query i=1	→ dp[1]=14
Add ℓ_1	Deque $= [\ell_0, \ell_1]$
Query i=2	→ dp[2]=19
Add ℓ_2	Deque $= [\ell_0, \ell_1, \ell_2]$
Query i=3	pop ℓ_0, ℓ_1 → dp[3]=38
Add ℓ_3	Deque $= [\ell_2, \ell_3]$
Query i=4	pop ℓ_2 → dp[4]=52
Add ℓ_4	Deque $= [\ell_3, \ell_4]$
Query i=5	→ dp[5]=64

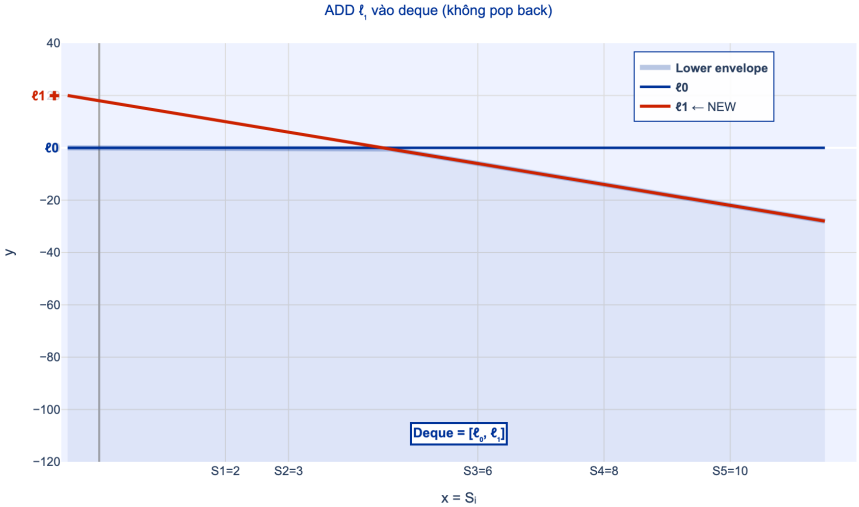
Deque – Bước 1: ADD ℓ_0



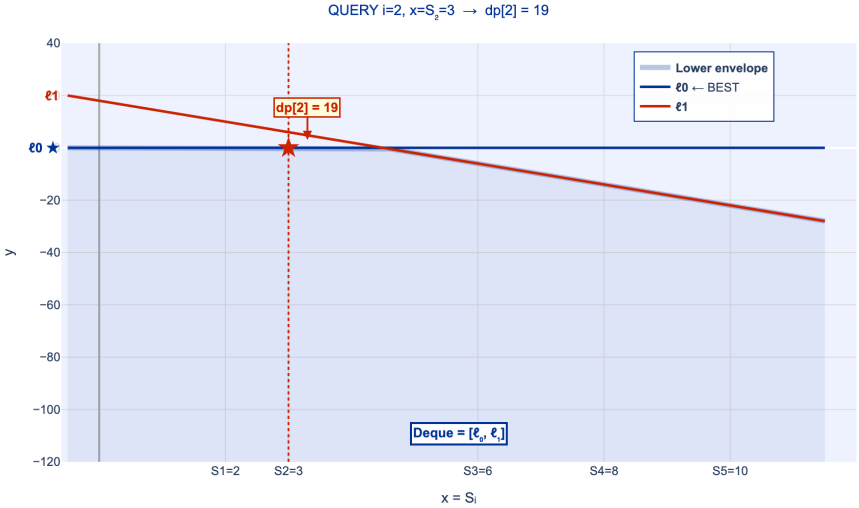
Deque – Bước 2: QUERY $i = 1$, $x = 2$



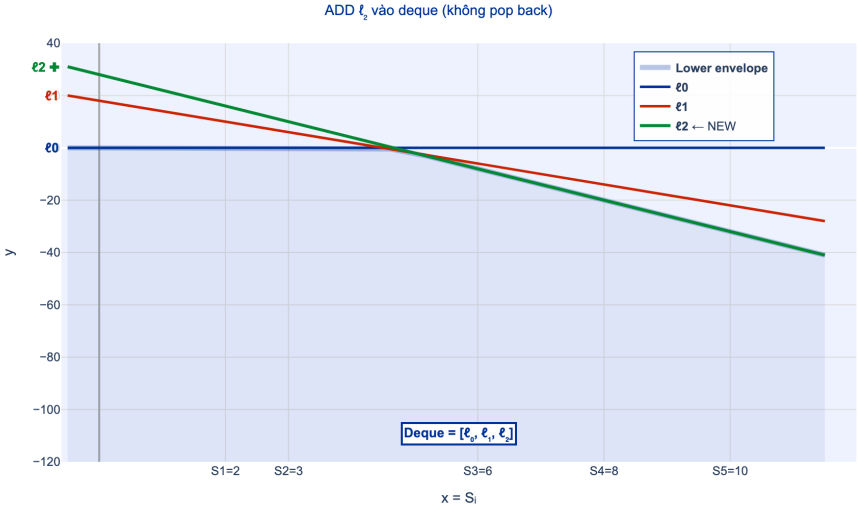
Deque – Bước 3: ADD ℓ_1



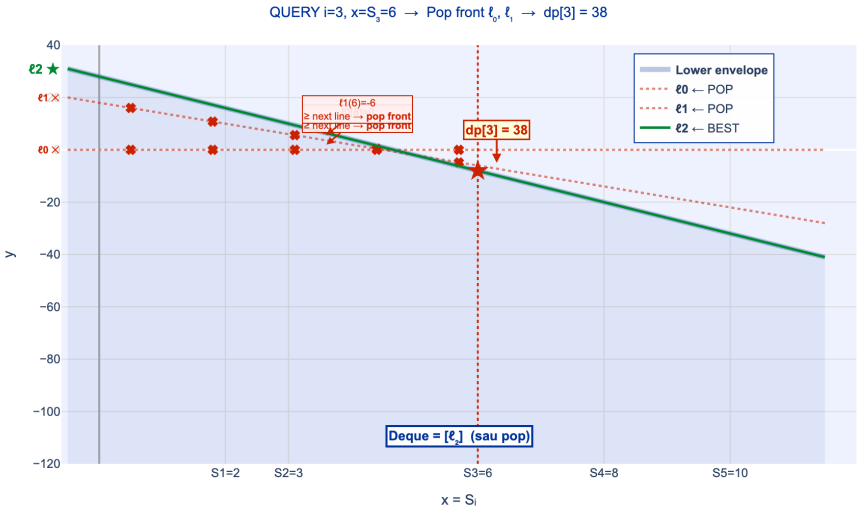
Deque – Bước 4: QUERY $i = 2$, $x = 3$



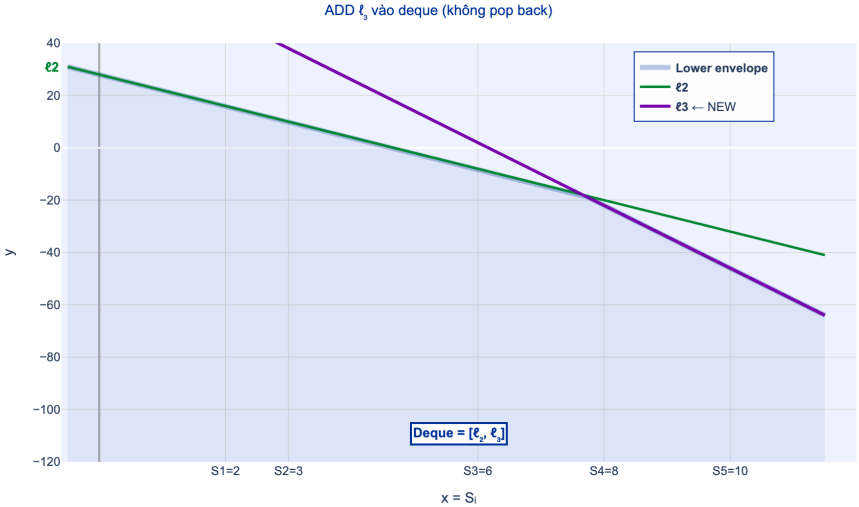
Deque – Bước 5: ADD ℓ_2



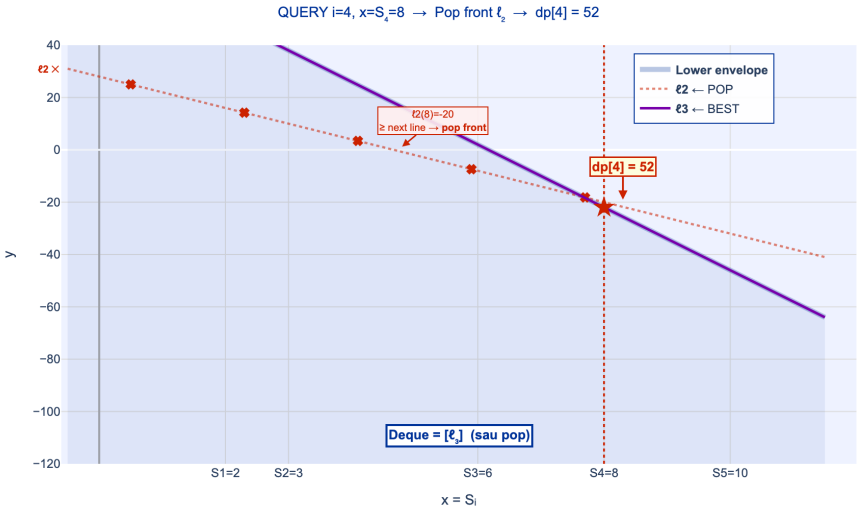
Deque – Bước 6: QUERY $i = 3$, $x = 6$ (Pop front ℓ_0, ℓ_1)



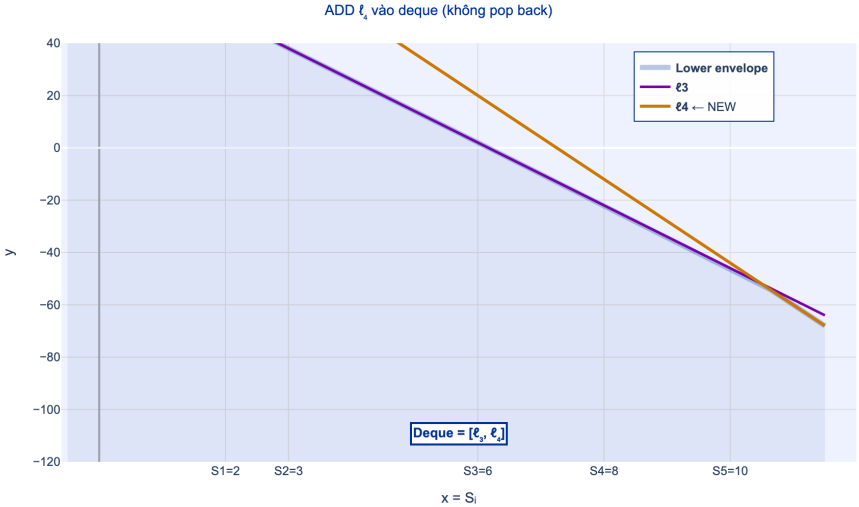
Deque – Bước 7: ADD ℓ_3



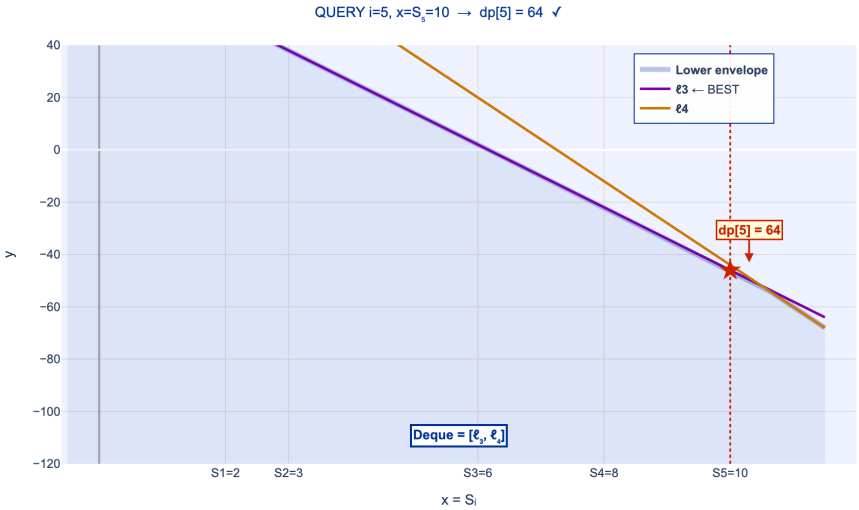
Deque – Bước 8: QUERY $i = 4$, $x = 8$ (Pop front ℓ_2)



Deque – Bước 9: ADD ℓ_4



Deque – Bước 10: QUERY $i = 5$, $x = 10$



Thuật toán CHT Deque – Pseudocode

Algorithm 1 CHT Deque (min, slope↓, query↑)

```
dq ← deque rỗng
function ADD(m, b)
    while |dq| ≥ 2 and bad(dq[−2], dq[−1], (m, b)) do
        dq.pop_back()
    end while
    dq.push_back((m, b))
end function
function QUERY(x)
    while |dq| ≥ 2 and dq[0](x) ≥ dq[1](x) do
        dq.pop_front()
    end while
    return dq[0](x)
end function
```

Hàm bad(l_1, l_2, l_3)

l_2 bị loại khi:

$$\frac{b_3 - b_1}{m_1 - m_3} \leq \frac{b_2 - b_1}{m_1 - m_2}$$

Thuật toán CHT Deque – Pseudocode

Algorithm 2 CHT Deque (min, slope↓, query↑)

```
dq ← deque rỗng
function ADD(m, b)
    while |dq| ≥ 2 and bad(dq[−2], dq[−1], (m, b)) do
        dq.pop_back()
    end while
    dq.push_back((m, b))
end function
function QUERY(x)
    while |dq| ≥ 2 and dq[0](x) ≥ dq[1](x) do
        dq.pop_front()
    end while
    return dq[0](x)
end function
```

Hàm bad(l_1, l_2, l_3)

l_2 bị loại khi:

$$\frac{b_3 - b_1}{m_1 - m_3} \le \frac{b_2 - b_1}{m_1 - m_2}$$

$$\Updownarrow$$

$$(b_3 - b_1)(m_1 - m_2) \le (b_2 - b_1)(m_1 - m_3)$$

Cài đặt C++ – Bài Mở quà (1/2)

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef __int128 lll;

struct Line { ll m, b; };

ll eval(Line l, ll x) { return l.m * x + l.b; }

// l2 bi loai neu giao(l1,l3) <= giao(l1,l2)
bool bad(Line l1, Line l2, Line l3) {
    return (lll)(l3.b - l1.b) * (l1.m - l2.m)
        <= (lll)(l2.b - l1.b) * (l1.m - l3.m);
}

deque<Line> dq;

void addLine(ll m, ll b) {
    Line nl = {m, b};
    while (dq.size() >= 2 && bad(dq[dq.size()-2], dq.back(), nl))
        dq.pop_back();
    dq.push_back(nl);
}
```

`__int128`: tránh tràn số khi nhân hai số $\leq 10^{18}$.

Cài đặt C++ – Bài Mở quà (2/2)

```cpp
ll query(ll x) {
    while (dq.size() >= 2 && eval(dq[0], x) >= eval(dq[1], x))
        dq.pop_front();
    return eval(dq[0], x);
}

int main() {
    int n; cin >> n;
    vector<ll> a(n+1), s(n+1);
    for (int i = 1; i <= n; i++) { cin >> a[i]; s[i] = s[i-1] + a[i]; }

    vector<ll> dp(n+1, 1e18);
    dp[0] = 0;
    addLine(0, 0);            // j=0: m=0, b=dp[0]+s[0]^2=0

    for (int i = 1; i <= n; i++) {
        dp[i] = s[i]*s[i] + query(s[i]);
        addLine(-2*s[i], dp[i] + s[i]*s[i]);
    }
    cout << dp[n] << "\n";
}
```

Độ phức tạp – Bài Mở quà

Thuật toán	Thời gian	Bộ nhớ	Kết quả
Naive vòng lặp đôi	$\mathcal{O}(N^2)$	$\mathcal{O}(N)$	**TLE**
CHT + Deque	$\mathcal{O}(N)$	$\mathcal{O}(N)$	**AC**

Tại sao deque đạt $\mathcal{O}(N)$?

- Mỗi đường thẳng được **push** vào deque đúng **1 lần**.
- Mỗi đường thẳng được **pop** ra khỏi deque nhiều nhất **1 lần**.
- Tổng số thao tác push + pop $= \mathcal{O}(N)$.

So sánh nhanh

$N = 2 \times 10^5$: $\mathcal{O}(N^2) \approx 4 \times 10^{10}$ phép vs $\mathcal{O}(N) \approx 2 \times 10^5$ phép ($\times 200\,000$ lần nhanh hơn)

Ví dụ chạy tay – $N = 4$, $a = [1, 2, 3, 4]$

$S = [0, 1, 3, 6, 10]$. Đường ℓ_j: $m_j = -2S_j$, $b_j = \mathrm{dp}[j] + S_j^2$.

i	$x = S_i$	Line thêm (m_j, b_j)	Deque	$\mathrm{dp}[i] = S_i^2 + \min \ell_j(S_i)$
–	–	$(0,\ 0)$	$[(0, 0)]$	$\mathrm{dp}[0] = 0$
1	1	$(-2,\ 2)$	$[(0, 0),\ (-2, 2)]$	$1 + \ell_0(1) = 1 + 0 = \mathbf{1}$
2	3	$(-6,\ 14)$	$[(-2, 2),\ (-6, 14)]$	$9 + \ell_1(3) = 9 + (-4) = \mathbf{5}$
3	6	$(-12,\ 50)$	$[(-6, 14),\ (-12, 50)]$	$36 + \ell_2(6) = 36 + (-22) = \mathbf{14}$
4	10	–	$[(-12, 50)]$	$100 + \ell_3(10) = 100 + (-70) = \mathbf{30}$

Đáp án: $\mathrm{dp}[4] = \mathbf{30}$

Ví dụ chạy tay 2 – $N = 5$, $w = [2, 1, 3, 2, 2]$, $C = 10$

Công thức với hằng số C

$$\mathrm{dp}[i] = S_i^2 + C + \min_{j<i}\{\underbrace{(-2S_j)}_{m_j} \cdot S_i + \underbrace{\mathrm{dp}[j] + S_j^2}_{b_j}\} \quad \text{với } S = [0, 2, 3, 6, 8, 10]$$

i	$x{=}S_i$	Line thêm $(m_j,\ b_j)$	Deque (sau thao tác)	$\mathrm{dp}[i] = S_i^2 + C + \min \ell_j(S_i)$
–	–	$(0,\ 0)$	$[(0,0)]$	$\mathrm{dp}[0] = 0$
1	2	$(-4,\ 18)$	$[(0,0),\ (-4,18)]$	$4 + 10 + \ell_0(2) = 4 + 10 + 0 = \mathbf{14}$
2	3	$(-6,\ 28)$	$[(0,0),\ (-4,18),\ (-6,28)]$	$9 + 10 + \ell_0(3) = 9 + 10 + 0 = \mathbf{19}$
3	6	$(-12,\ 74)$	$\xrightarrow{\text{pop } \ell_0, \ell_1} [(-6,28),\ (-12,74)]$	$36 + 10 + \ell_2(6) = 36 + 10 + (-8) = \mathbf{38}$
4	8	$(-16,\ 116)$	$\xrightarrow{\text{pop } \ell_2} [(-12,74),\ (-16,116)]$	$64 + 10 + \ell_3(8) = 64 + 10 + (-22) = \mathbf{52}$
5	10	–	$[(-12,74),\ (-16,116)]$	$100 + 10 + \ell_3(10) = 100 + 10 + (-46) = \mathbf{64}$

Đáp án: $\mathrm{dp}[5] = \mathbf{64}$

Lưu ý

$i = 3$: pop **2 lần** liên tiếp $(\ell_0,\ \ell_1)$ trước khi query. C **không** ảnh hưởng đến slope/intercept.

Bài 2: Mỏ quặng – Đề bài

Bài toán

Có n mỏ với sản lượng $x_1, \ldots, x_n$. Chia toàn bộ dãy thành các cụm **liên tiếp** không giao nhau. Cụm $[l..r]$ có sản lượng $S = \sum_{i=l}^{r} x_i$, đóng góp $aS^2 + bS + c$. $(a < 0)$

Tối đa hoá tổng sản lượng điều chỉnh.

Recurrence

Đặt $P_i = x_1 + \cdots + x_i$ (prefix sum), $P_0 = 0$.

$$\mathrm{dp}[i] = \max_{0 \le j < i} \left\{ \mathrm{dp}[j] + a(P_i - P_j)^2 + b(P_i - P_j) + c \right\}$$

Ví dụ

$n=5$, $a=-1$, $b=8$, $c=-5$
$x=[1, 2, 3, 2, 1]$, $P=[0, 1, 3, 6, 8, 9]$

Chia $[1\,2]\,[3]\,[2\,1]$: mỗi cụm $S=3$
$\Rightarrow$ mỗi cụm $= -9+24-5 = 10$
Tổng = 30

Ràng buộc: $n \le 10^6$, $-5 \le a \le -1$, $|b|, |c| \le 10^7$, $1 \le x_i \le 100$

Bài 2 – Khai triển đại số

Mở rộng bình phương

$$\begin{aligned}\mathrm{dp}[i] &= \max_j \{\mathrm{dp}[j] + a(P_i - P_j)^2 + b(P_i - P_j) + c\} \\ &= aP_i^2 + bP_i + c + \max_j \{\underbrace{\mathrm{dp}[j] + aP_j^2 - bP_j}_{b_j} + \underbrace{(-2a\,P_j)}_{m_j} \cdot P_i\}\end{aligned}$$

Đường thẳng ℓ_j

$$\ell_j : \; y = \underbrace{(-2a\,P_j)}_{m_j} x + \underbrace{(\mathrm{dp}[j] + aP_j^2 - bP_j)}_{b_j}$$

$\mathrm{dp}[i] = aP_i^2 + bP_i + c + \max_j\{\ell_j(P_i)\}$

Tính đơn điệu slope

$m_j = -2a\,P_j$.
$a < 0 \Rightarrow -2a > 0$, P_j tăng:

$$m_0 \le m_1 \le m_2 \le \cdots$$

Slope **tăng dần** + query tăng dần
$\Rightarrow$ **Max CHT Deque O(N)**

Min CHT vs Max CHT – Chỉ **một** điều thay đổi

Min CHT (Mở quà)

- Tìm $\min_j$, slope **giảm dần**
- bad: $(b_3-b_1)(m_1-m_2) \le (b_2-b_1)(m_1-m_3)$
- Pop front khi $\ell_0(x) \ge \ell_1(x)$

Max CHT (Mỏ quặng)

- Tìm $\max_j$, slope **tăng dần**
- bad: $(b_3-b_1)(m_1-m_2) \le (b_2-b_1)(m_1-m_3)$
- Pop front khi $\ell_0(x) \le \ell_1(x)$

Min CHT vs Max CHT – Chỉ **một** điều thay đổi

Min CHT (Mở quà)

- Tìm $\min_j$, slope **giảm dần**
- bad: $(b_3-b_1)(m_1-m_2) \le (b_2-b_1)(m_1-m_3)$
- Pop front khi $\ell_0(x) \ge \ell_1(x)$

Max CHT (Mỏ quặng)

- Tìm $\max_j$, slope **tăng dần**
- bad: $(b_3-b_1)(m_1-m_2) \le (b_2-b_1)(m_1-m_3)$
- Pop front khi $\ell_0(x) \le \ell_1(x)$

Tóm tắt so sánh

	Min CHT	**Max CHT**
Envelope	Bao dưới	Bao trên
Slope thêm	Giảm dần	**Tăng dần**
bad()	**Giống hệt** — $(b_3-b_1)(m_1-m_2) \le (b_2-b_1)(m_1-m_3)$	
Pop front	$\ell_0(x) \ge \ell_1(x)$	$\ell_0(x) \le \ell_1(x)$

bad() dùng **cùng một công thức** cho cả hai bài. Chỉ **pop front** đổi dấu vì một bài tìm $\min$, một bài tìm $\max$.

Chứng minh: bad() của Min CHT và Max CHT giống hệt nhau

Định nghĩa hình học: ℓ_2 bị loại

ℓ_2 không bao giờ là tốt nhất $\iff$ giao (ℓ_1, ℓ_3) xảy ra **trước hoặc cùng lúc** giao (ℓ_1, ℓ_2):

$$\underbrace{\frac{b_1 - b_3}{m_3 - m_1}}_{x^*(\ell_1,\ell_3)} \le \underbrace{\frac{b_1 - b_2}{m_2 - m_1}}_{x^*(\ell_1,\ell_2)}$$

Khai triển đại số (nhân chéo — mẫu cùng dấu trong mỗi trường hợp)

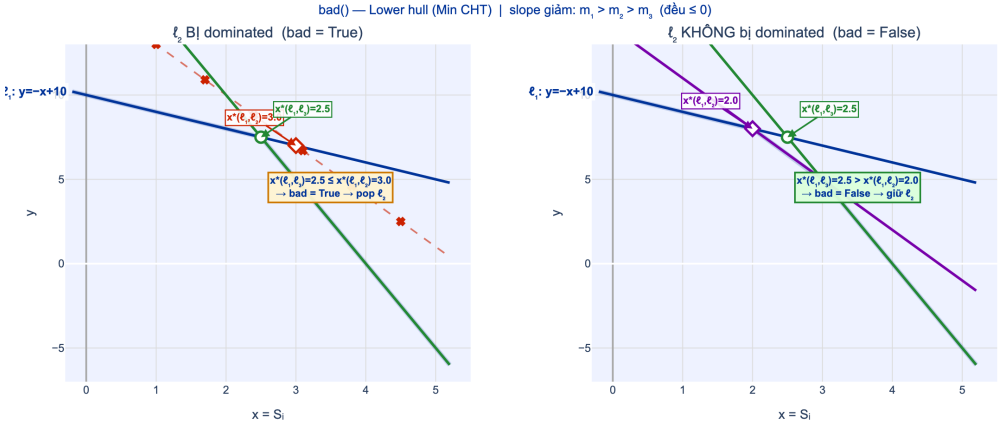
$$(b_1 - b_3)(m_2 - m_1) \le (b_1 - b_2)(m_3 - m_1)$$

$$\Updownarrow \quad \text{(viết lại: } b_1 - b_k = -(b_k - b_1),\ m_k - m_1 = -(m_1 - m_k)\text{)}$$

$$\underbrace{(b_3 - b_1)(m_1 - m_2)}_{\texttt{lhs}} \le \underbrace{(b_2 - b_1)(m_1 - m_3)}_{\texttt{rhs}}$$

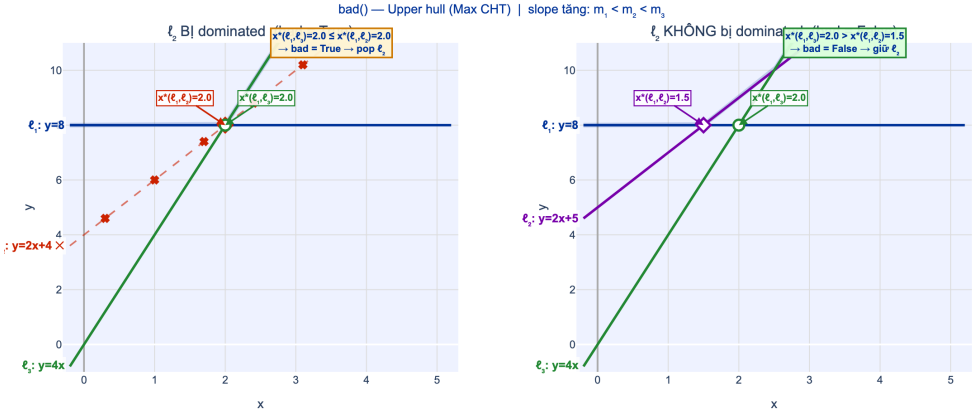
$$\boxed{\texttt{bad}(\ell_1, \ell_2, \ell_3) \equiv \texttt{lhs} \le \texttt{rhs} \quad \text{— cùng một dấu <= cho cả hai bài!}}$$

Minh hoạ bad() – Lower hull (Min CHT)



ℓ_2 bị dominated $\Leftrightarrow x^*(\ell_1, \ell_3) \leq x^*(\ell_1, \ell_2)$ — ℓ_3 "cắt" ℓ_1 trước khi ℓ_2 kịp cắt.

Minh hoạ bad() – Upper hull (Max CHT)



Cùng điều kiện $x^*(\ell_1, \ell_3) \le x^*(\ell_1, \ell_2)$, cùng công thức `lhs <= rhs`. Chỉ đổi chiều đường bao (dưới $\leftrightarrow$ trên), **không đổi** `bad()`.

Cài đặt C++ – Bài Mỏ quặng

```cpp
// slope tang dan (m_j = -2a*P[j], a<0)
bool bad_max(Line l1, Line l2, Line l3) {
    return (__int128)(l1.b-l3.b)*(l2.m-l1.m)
        >= (__int128)(l1.b-l2.b)*(l3.m-l1.m);
}
ll query_max(ll x) {
    while (dq.size()>=2 && eval(dq[0],x) <= eval(dq[1],x))
        dq.pop_front();
    return eval(dq[0], x);
}
// dp[i] = a*P[i]^2 + b*P[i] + c + query_max(P[i])
// m_j = -2a*P[j],  b_j = dp[j] + a*P[j]^2 - b*P[j]
for (int i = 1; i <= n; i++) {
    dp[i] = (ll)a*P[i]*P[i] + b*P[i] + c + query_max(P[i]);
    ll mj = -2LL*a*P[i];
    ll bj = dp[i] + (ll)a*P[i]*P[i] - (ll)b*P[i];
    addLine_max(mj, bj);
}
```

Ví dụ chạy tay – $n{=}5$, $a{=}-1$, $b{=}8$, $c{=}-5$, $x{=}[1,2,3,2,1]$

$P=[0,1,3,6,8,9]$. $m_j=2P_j$. $b_j=\mathrm{dp}[j]-P_j^2-8P_j$.

i	P_i	Line thêm (m_j,b_j)	Deque (sau thao tác)	$aP_i^2{+}bP_i{+}c$	$\mathrm{dp}[i]$
–	0	$(0,\ 0)$	$[\ell_0{=}(0,0)]$	–	0
1	1	$(2,\ -7)$	$[\ell_0,\ \ell_1]$	2	$2{+}\ell_0(1)=2{+}0=\mathbf{2}$
2	3	$(6,\ -23)$	$\xrightarrow{\text{pop back } \ell_1}[\ell_0,\ \ell_2]$	10	$10{+}\ell_0(3)=10{+}0=\mathbf{10}$
3	6	$(12,\ -64)$	$\xrightarrow{\text{pop front } \ell_0}[\ell_2,\ \ell_3]$	7	$7{+}\ell_2(6)=7{+}13=\mathbf{20}$
4	8	$(16,\ -101)$	$\xrightarrow{\text{pop front } \ell_2}[\ell_3,\ \ell_4]$	-5	$-5{+}\ell_3(8)=-5{+}32=\mathbf{27}$
5	9	–	$[\ell_3,\ \ell_4]$	-14	$-14{+}\ell_3(9)=-14{+}44=\mathbf{30}$

Đáp án: $\mathrm{dp}[5]=\mathbf{30}$ ✓

Điểm đáng chú ý

$i{=}2$: **pop back** ℓ_1 (ℓ_1 bị ℓ_0,ℓ_2 che).
$i{=}3,4$: **pop front** (query tăng, đầu deque lỗi thời).

Bài 3: Ván gỗ – Đề bài

Bài toán

Có n tấm ván (w_i, l_i) (chiều rộng, chiều dài). Mỗi lần mua một **nhóm**, chi phí $= \max(w) \times \max(l)$ trong nhóm. **Tối thiểu hoá** tổng chi phí mua hết n tấm.

Ví dụ: $n = 3$

Tấm	w_i	l_i
1	2	5
2	4	3
3	6	2

Chia: $\{3\}$, $\{2\}$, $\{1\}$

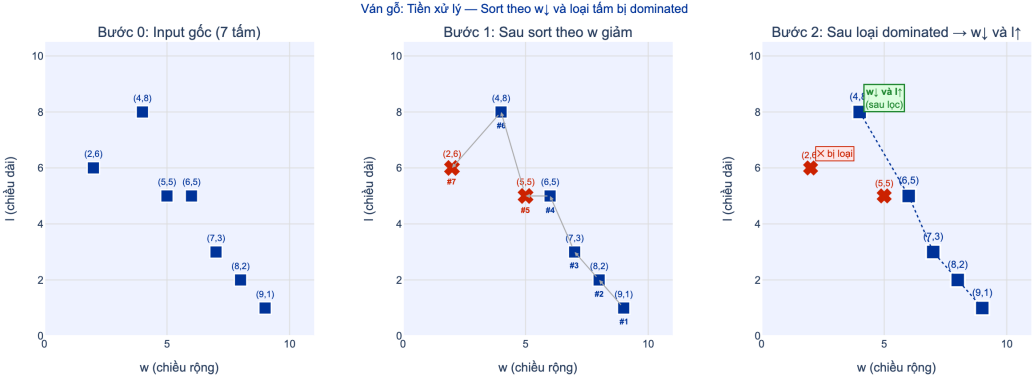
$= 6{\cdot}2 + 4{\cdot}3 + 2{\cdot}5 = 12 + 12 + 10 = \mathbf{28}$

Ràng buộc

$1 \le n \le 10^5, \;\; 1 \le w_i, l_i \le 10^6$

Cần $\mathcal{O}(n \log n)$ hoặc $\mathcal{O}(n)$.

Bài 3 – Minh hoạ: Sort và loại tấm bị dominated



Sau loại dominated: w giảm **đồng thời** l tăng $\Rightarrow$ slope và query đều đơn điệu $\Rightarrow$ CHT Deque.

Bài 3 – Bước 1: Sort + Loại tấm bị dominated

Sort theo w giảm dần

Sau sort: $w_1 \geq w_2 \geq \cdots \geq w_n$. Với bất kỳ cách chia nào, $\max(w)$ của mỗi nhóm là w của tấm **đứng đầu nhóm** (tấm có w lớn nhất). $\Rightarrow$ Tối ưu chỉ cần chia thành các **đoạn liên tiếp** sau sort.

Loại tấm bị dominated

Tấm j bị dominated nếu tồn tại tấm i đứng trước ($w_i \geq w_j$) với $l_i \geq l_j$: tấm j không bao giờ cần là $\max(l)$. $\Rightarrow$ Loại tấm j khỏi xét (gộp vào nhóm của tấm i).

Kết quả sau hai bước tiền xử lý

Còn lại m tấm với $w_1 > w_2 > \cdots > w_m$ **và** $l_1 < l_2 < \cdots < l_m$ (đồng thời tăng – giảm).

Bài 3 – Bước 2: DP trên dãy đã lọc

Recurrence

Sau lọc: w giảm, l tăng. Nhóm $[j+1 \dots i]$ có:

$$\max(w) = w_{j+1}, \quad \max(l) = l_i \text{ (vì } l \text{ tăng)}$$

$$\mathrm{dp}[i] = \min_{0 \le j < i} \{\mathrm{dp}[j] + w_{j+1} \cdot l_i\}, \quad \mathrm{dp}[0] = 0$$

Nhận ra dạng CHT

$$\mathrm{dp}[i] = \min_j \{\underbrace{w_{j+1}}_{m_j} \cdot \underbrace{l_i}_{x} + \underbrace{\mathrm{dp}[j]}_{b_j}\}$$

- $m_j = w_{j+1}$: slope **giảm dần** (w giảm sau lọc)
- $x_i = l_i$: query **tăng dần** (l tăng sau lọc)

$\Rightarrow$ **Min CHT Deque** $\mathcal{O}(n)$ sau tiền xử lý $\mathcal{O}(n \log n)$

Bài 3 – Ví dụ chạy tay

Input: $(w, l) = [(2, 5), (4, 3), (6, 2)]$

Sau sort + lọc

Sort $w \downarrow$: $[(6, 2), (4, 3), (2, 5)]$
Không tấm nào bị dominated

(w giảm, l tăng tự nhiên ✓)

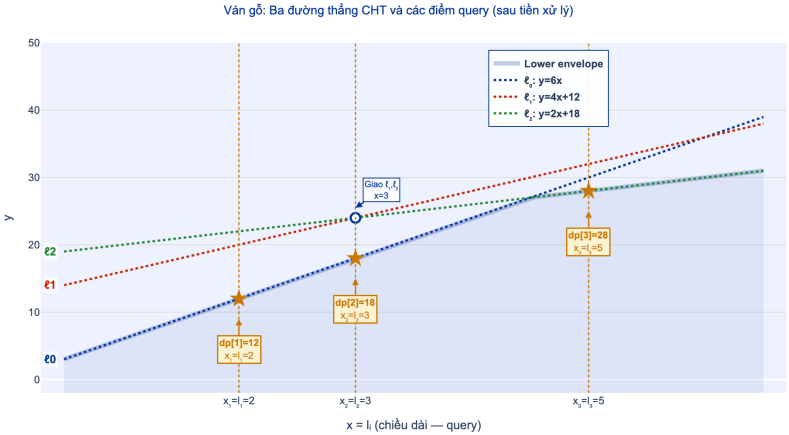
j	w_j	l_j
1	6	2
2	4	3
3	2	5

Đường thẳng ℓ_j: $y = w_{j+1} \cdot x + \mathrm{dp}[j]$

j	$m_j = w_{j+1}$	$b_j = \mathrm{dp}[j]$	query $x = l_i$	$\mathrm{dp}[i]$
0	6	0	$x_1{=}2$	$6{\cdot}2{+}0 = \mathbf{12}$
1	4	12	$x_2{=}3$	$\min(18, 24) = \mathbf{18}$
2	2	18	$x_3{=}5$	$\min(30, 32, 28) = \mathbf{28}$

Đáp án: $\mathrm{dp}[3] = \mathbf{28}$

Bài 3 – Minh hoạ: Ba đường thẳng CHT



Slope giảm $(6 > 4 > 2)$, query tăng $(x = 2 < 3 < 5)$ $\Rightarrow$ đường tối ưu dịch phải theo query $\Rightarrow$ **Min CHT Deque O(n)**.

Cài đặt C++ – Bài Ván gỗ

```cpp
int n; cin >> n;
vector<pair<ll,ll>> v(n);
for (auto& [w,l] : v) cin >> w >> l;

// Buoc 1: sort theo w giam, neu bang thi l giam
sort(v.begin(), v.end(), [](auto& a, auto& b){
    return a.first != b.first ? a.first > b.first : a.second > b.second;
});

// Buoc 2: loai dominated (giu lai thi l tang dan)
vector<pair<ll,ll>> a; // (w, l) da loc
for (auto [w,l] : v) {
    while (!a.empty() && a.back().second <= l)
        a.pop_back();  // tam bi dominated
    a.push_back({w, l});
}
// a[0].w > a[1].w > ... &&& a[0].l < a[1].l < ...

// Buoc 3: Min CHT Deque
int m = a.size();
vector<ll> dp(m+1, 0);
deque<pair<ll,ll>> dq; // (slope=w[j+1], intercept=dp[j])
dq.push_back({a[0].first, 0});

for (int i = 1; i <= m; i++) {
    ll x = a[i-1].second;  // query = l[i]
    while (dq.size()>=2 && dq[1].first*x+dq[1].second <= dq[0].first*x+dq[0].second)
        dq.pop_front();
    dp[i] = dq.front().first * x + dq.front().second;
    if (i < m)
        dq.push_back({a[i].first, dp[i]});
}
cout << dp[m] << "\n";
```

Bài 3 – Tóm tắt: Vai trò tiền xử lý

Tại sao cần 2 bước tiền xử lý?

1. **Sort theo** $w \downarrow$: đảm bảo tối ưu chỉ cần chia đoạn liên tiếp.
2. **Loại dominated**: biến dãy thành $w \downarrow$ **VÀ** $l \uparrow \Rightarrow$ slope *và* query đều đơn điệu $\Rightarrow$ CHT Deque được.

So sánh với Bài 1 (Mở quà)

	Mở quà	**Ván gỗ**
Slope tự nhiên đơn điệu?	Có ($-2S_j$ giảm)	**Không** – cần tiền xử lý
Query tự nhiên đơn điệu?	Có (S_i tăng)	**Không** – cần tiền xử lý
Sau tiền xử lý	–	Slope & query đều đơn điệu
Cấu trúc	Deque $\mathcal{O}(n)$	Deque $\mathcal{O}(n)$ (sau $\mathcal{O}(n \log n)$)

Bài 4: Tia và đường thẳng – Đề bài

Bài toán

Cho N tia (half-line), tia i có phương trình $y = m_i x + b_i$ hợp lệ với $x \geq x_{\min,i}$. Cho M query điểm $q_1, \ldots, q_M$. Với mỗi query q_k, tìm tia cho giá trị y **nhỏ nhất** tại $x = q_k$.

Điểm khác biệt cơ bản

Đây không phải DP!
Đây là bài toán hình học thuần tuý trên tập tia – không có cấu trúc $\mathrm{dp}[j] \to \mathrm{dp}[i]$ nào cả.

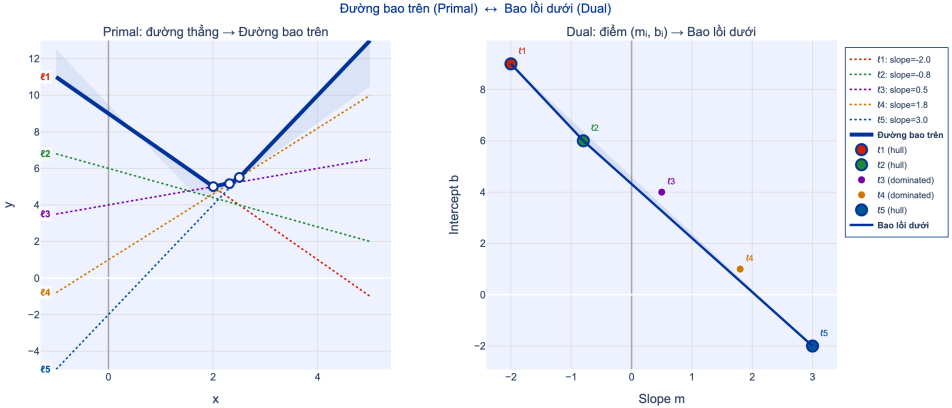
Ràng buộc

$N, M \leq 2 \times 10^5$

Cần $\mathcal{O}((N + M) \log N)$.

Bài này là ví dụ cho thấy CHT không chỉ là "mẹo DP" mà là **cách nhìn tập đường thẳng** – áp dụng được trong nhiều bài hình học.

Đường bao lồi – Góc nhìn hình học



Primal: đường thẳng $y = mx + b \longleftrightarrow$ **Dual**: điểm (m, b)

Đường bao trên (Upper Envelope) $\longleftrightarrow$ Bao lồi dưới (Lower Convex Hull)

Bao lồi dưới = danh sách đoạn thẳng

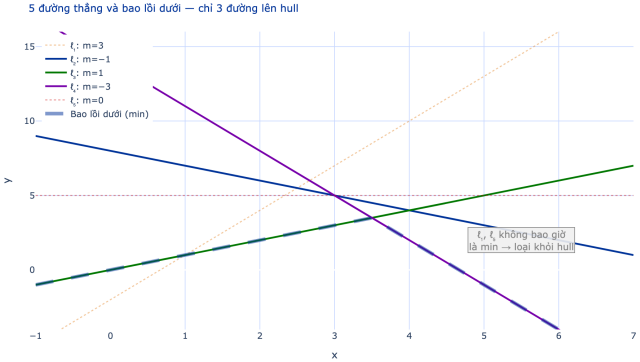
Sau khi build hull

N đường thẳng $\to$ chỉ còn $K \le N$ đường trên hull,
sắp theo slope tăng dần.

Trục x bị chia thành K **đoạn liên tiếp**.
Mỗi đoạn có đúng **một đường thẳng**.

Ví dụ: 5 đường $\to$ hull gồm 3

- ℓ_1, ℓ_5: không bao giờ là min $\to$ loại.
- Hull:
  ℓ_4 $(m{=}{-3})$, ℓ_2 $(m{=}{-1})$, ℓ_3 $(m{=}1)$.
- Ba vùng x tương ứng, ranh giới là giao điểm liên tiếp.



Mỗi đoạn x – một đường thẳng

Cấu trúc hull sau khi build

Hull lưu các đường thẳng **theo slope tăng**.

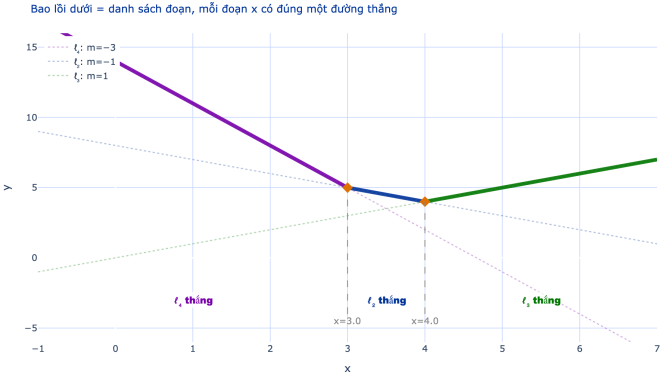
Điểm giao giữa ℓ_i và ℓ_{i+1} là **ranh giới** chuyển đường thẳng.

Gọi ranh giới thứ i là x_i^*:

$$x_1^* < x_2^* < \cdots < x_{K-1}^*$$

Tính chất quan trọng

Vùng thắng của mỗi đường là một **đoạn liên tục, không rời rạc**. Đây là chìa khoá cho two-pointer.



Tại sao two-pointer hoạt động?

Lập luận

1. Sort query tăng dần: $q_1 \le q_2 \le \cdots \le q_M$.
2. Với q_k, đường thẳng nằm ở vùng $[x^*_{p-1}, x^*_p]$ nào đó, với pointer p.
3. Với $q_{k+1} \ge q_k$, đường thẳng **không thể ở vùng trước** p (vì ranh giới đã qua).

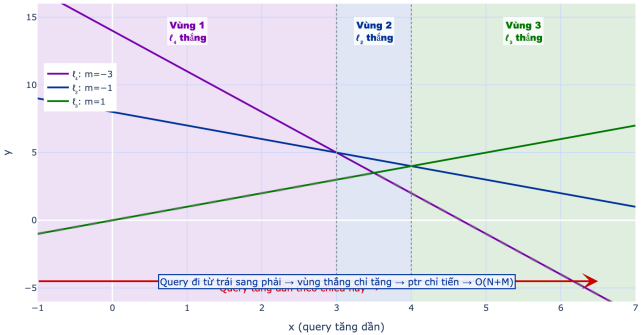
$\Rightarrow$ Pointer p **chỉ tiến**, không lùi.

Tổng số bước

Pointer đi qua mỗi cạnh hull *nhiều nhất một lần*

$\Rightarrow$ tổng bước two-pointer: $\mathcal{O}(N + M)$.

Tại sao two-pointer đúng: mỗi query chỉ nằm trong đúng một vùng liên tiếp



Two-pointer từng bước – Bước 1

Query $q_1 = 0.5$

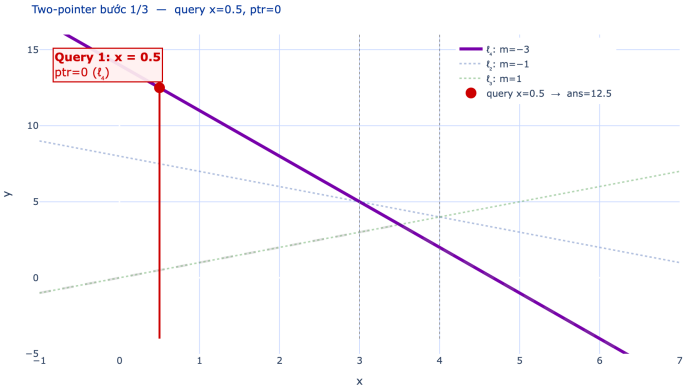
ptr $= 0$ (đang ở ℓ_4, vùng trái nhất).

Kiểm tra: hull[0](0.5) $\leq$ hull[1](0.5)?

$$\ell_4(0.5) = -1.5 < \ell_2(0.5) = 7.5$$

$\Rightarrow$ ptr không tiến.

Trả lời: $\ell_4(0.5) = -1.5$.



Two-pointer từng bước – Bước 2

Query $q_2 = 2.5$

Kiểm tra: hull[0](2.5) $\leq$ hull[1](2.5)?

$\ell_4(2.5) = 6.5 > \ell_2(2.5) = 5.5$

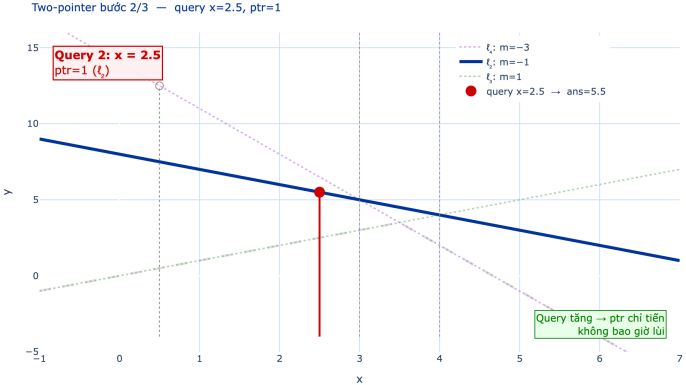
$\Rightarrow$ ptr tiến lên 1.

Kiểm tra tiếp hull[1] vs hull[2]:

$\ell_2(2.5) = 5.5 < \ell_3(2.5) = 2.5$?

Không $\Rightarrow$ dừng.

Trả lời: $\ell_2(2.5) = 5.5$.



Two-pointer từng bước – Bước 3

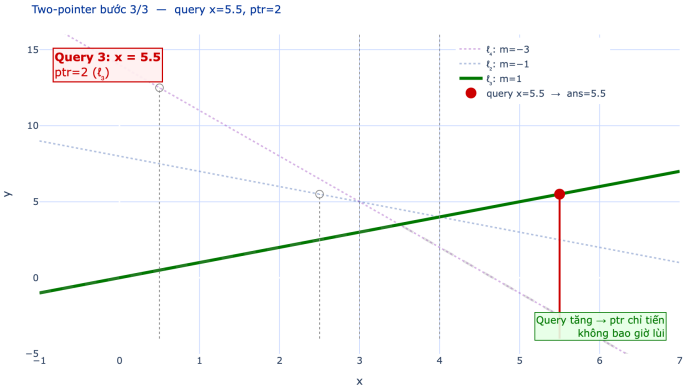
Query $q_3 = 5.5$

Kiểm tra: hull$[1](5.5) \leq$ hull$[2](5.5)$?
$\ell_2(5.5) = 2.5 > \ell_3(5.5) = 5.5$?
Không – ℓ_3 thắng $\Rightarrow$ ptr tiến.

hull$[2]$ là đường cuối $\Rightarrow$ dừng.
Trả lời: $\ell_3(5.5) = 5.5$.

Tổng kết 3 bước

ptr: $0 \to 1 \to 2$ – **chỉ tiến**.
Tổng bước: 2 lần tiến + 3 query
$= \mathcal{O}(N + M)$.



Xử lý Offline – Sắp xếp truy vấn

Algorithm 3 Offline: Hull + Two-pointer

```
Build lower envelope (N đường, sort + lọc)
Sort M query theo x tăng (lưu index)
ptr ← 0
for all (x_k, idx_k) tăng dần do
    while ptr + 1 < |hull| and hull[ptr](x_k) ≥ hull[ptr + 1](x_k) do
        ptr += 1
    end while
    ans[idx_k] ← hull[ptr](x_k)
end for
In ans[1 ... M] theo thứ tự gốc
```

Độ phức tạp

- Build hull: $\mathcal{O}(N \log N)$
- Sort query: $\mathcal{O}(M \log M)$
- Two-pointer: $\mathcal{O}(N + M)$

Tổng: $\mathcal{O}((N + M) \log(N + M))$